

Table of Contents

Plan de securisation et d'observabilite de Claviger

Version analysee : branche main fournie le 7 septembre 2026

Contexte cible : bot Discord V1, SQLite, futur deploiement Docker sur un mini-PC.

1. Resume

Claviger ne presente pas, dans le code fourni, de secret en clair, d'injection SQL evidente ou de controle d'administration manifestement ouvert a tous. Les commandes administratives principales sont reservees au proprietaire du serveur, .env et les fichiers DB sont ignores par Git, et les requetes SQLite parametrent les valeurs.

La priorite de securisation n'est donc pas de multiplier les outils. Elle est de fermer les ecarts entre **etat attendu**, **etat reel** et **capacite a diagnostiquer un incident**.

Les risques les plus importants sont :

- erreurs inattendues non tracees dans certains parcours;
- absence de stack trace dans plusieurs reports;
- mutations Discord partiellement appliquees;
- incoherence de la determination d'un role gerable;
- identites fixes basees sur noms et cles logiques non uniques;
- future extension des capabilities sensibles a partir d'une confiance implicite par hierarchie;
- absence de verrouillage/reproductibilite des dependances et d'artefacts de production dans la branche auditee.

Aucun probleme **CRITIQUE** confirme n'a ete identifie. Plusieurs points **ELEVES/MOYENS** doivent etre traites avant ou juste apres le premier deploiement.

2. Matrice des risques

Severite	Zone	Probleme	Consequence	Recommanda tion
ELEVEE	Observabilite	Exceptions de certains workflows absorbees sans report	Incident invisible, diagnostic tardif	Error boundary + traceback + contexte structure
ELEVEE	Roles Discord	Echec au	Permissions	PartialExecut

Severite	Zone	Probleme	Consequence	Recommanda tion
		milieu d'une sequence de mutations	partiellement appliquees	ionError/ result + audit des mutations
MOYENNE	Catalogue	role_manage able ne filtre pas role.manage d dans discovery	Entree catalogue annoncee utilisable puis execution refusee	Predicat commun + test regression
MOYENNE	Identite	Roles/salons fixes resolu par nom	Ambiguite en cas de doublon	Resolution unique puis IDs persistants
MOYENNE	Integrite	Cles interest_key /access_key non uniques	Plusieurs roles pour une meme option logique	Detection + contrainte UNIQUE si invariant confirme
MOYENNE	Logging	Pas de configuration centrale du logging	INFO perdus, formats imprevisibles	Config runtime stdout structuree
MOYENNE	Reporting	str(error) sans traceback	Cause technique difficile a retrouver	Logger avec exc_info, forum sanitize
MOYENNE	Policy	Fallback Succumbrae sensible en panne DB	Futur fail- open si un consumer oublie le guard DB	SAFE_DEFAU LT en panne; bootstrap separe
MOYENNE	Supply chain	Pas de lock de dependances	Build Docker non reproductible	Lock versionne + CI
FAIBLE	/say	Politique de mentions implicite	Mentions de masse par utilisateur trusted	allowed_me ntions explicite/cap ability
FAIBLE	Reporting	Reporter	Reponse	Timeout

Severite	Zone	Probleme	Consequence	Recommanda tion
		Discord attendu synchronique ment	utilisateur retardee en panne Discord	borne, defer, local log prioritaire
FAIBLE	Rate limiting	Pas de cooldown applicatif visible	Spam de workflows / pression API	Ajouter seulement si usage reel le justifie
AMELIORATI ON	Monitoring	Pas de metriques/he alth applicatif	Vue limitee de la sante	Logs d'etat + checks legers avant stack metriques

3. Gestion des erreurs

3.1 Etat actuel

Le projet contient deja des exceptions specifiques dans plusieurs domaines, mais les boundaries ne les exploitent pas uniformement.

Exemples de captures generiques justifiees :

- ReportService.emit capture Exception autour de chaque reporter pour garantir l'isolation des destinations.

Exemples a ameliorer :

- member_command et MemberQuestionnaireModal capturent toutes les erreurs sans conserver la cause;
- NoctisQuestionnaireModal fait de meme;
- plusieurs UI catalogue renvoient seulement un message d'echec;
- les commandes admin emettent un ReportEvent, mais souvent avec uniquement str(error).

3.2 Architecture cible

Erreur technique basse couche

- > exception contextualisee
- > service/coordinator
- > boundary Discord
- > classification attendue/inattendue
- > log local complet (traceback)
- > report administratif sanitize si utile
- > reponse utilisateur neutre

3.3 Regles de capture

- Un repository doit traduire une erreur SQLite vers une exception de persistance contextualisee.
- Un planner doit lever une erreur de validation/metier, pas connaitre Discord.
- Un executor peut traduire les erreurs de l'API Discord en erreur d'execution contextualisee.
- Le command/modal boundary decide du message utilisateur et du niveau de log.
- Le traceback complet ne doit pas etre envoye dans le forum Discord.

4. Hierarchie d'exceptions recommandee

L'objectif n'est pas de remplacer toutes les classes en un commit. Une migration incrementale suffit.

ClavigerError

```
| - ConfigurationError
| - PersistenceError
|   | - DatabaseMissingError
|   | - DatabaseUnavailableError
|   ` - UnsupportedSchemaVersionError
| - ValidationError
|   | - UnknownSelectionError
|   ` - AmbiguousConfigurationError
| - WorkflowError
|   | - MemberWorkflowError
|   ` - NoctisWorkflowError
| - RoleExecutionError
|   | - RoleNotFoundError
|   | - RoleNotManageableError
|   ` - PartialRoleExecutionError
|- ExternalServiceError
  ` - ReportingDestinationError (si necessaire)
```

Les exceptions existantes peuvent progressivement heriter de bases communes sans changer leurs noms métier lorsque ceux-ci sont utiles.

PartialRoleExecutionError

Cette erreur doit transporter, au minimum :

- operation/workflow;
- role dont l'appel a echoue;
- action (add/remove);
- IDs deja ajoutes;

- IDs déjà retirés;
- type d'erreur Discord;
- indication si une compensation a été tentée.

Elle ne doit pas contenir le token ou des données de message utilisateur.

5. Politique de logs

Claviger doit distinguer deux notions :

1. **logs techniques** : destinés au diagnostic local/production;
2. **reports administratifs** : événements sémantiques suffisamment importants pour apparaître dans le forum ou une future UI.

Tout log technique n'est pas un ReportEvent.

DEBUG

Utilisation en développement ou diagnostic temporaire.

Exemples :

- nombre d'entrées catalogue chargées;
- nombre de rôles du snapshot;
- plan calculé en termes de compteurs, sans contenu sensible;
- temps d'une opération;
- choix du reporter configuré.

Éviter de logger le texte de messages utilisateurs ou les catégories adultes sélectionnées.

INFO

Événements normaux importants :

- démarrage du bot;
- commandes synchronisées;
- connexion/reconnexion Discord;
- DB prête et version de schéma;
- catalogue sync réussi avec compteurs;
- migration/initialisation réussie;
- bootstrap de guild;
- éventuel déploiement/version de build.

WARNING

Situation anormale mais gérée :

- tentative d'action sans permission;

- role catalogue ambigu/non gerable;
- mapping de channel invalide;
- policy manquante conduisant a safe default;
- reporter secondaire indisponible;
- tentative avec selection devenue obsolete puis refusee;
- rate limit applicatif si ajoute.

ERROR

Operation demandee qui echoue :

- erreur DB pendant une commande;
- erreur Discord pendant une mutation;
- echec d'ouverture/soumission d'un questionnaire pour cause technique;
- catalog sync rollback;
- configuration invalide detectee apres demarrage (idealement fail-fast avant).

Un ERROR inattendu doit conserver une stack trace localement.

CRITICAL

A reserver aux situations ou l'instance ne peut plus assurer sa fonction essentielle :

- impossibilite de demarrer avec configuration requise;
- corruption/incompatibilite de schema ne permettant aucun fonctionnement;
- bot incapable d'accéder durablement a une dependance indispensable et necessitant arret/intervention;
- invariant de securite fondamental viole.

Ne pas utiliser CRITICAL pour un simple echec de commande utilisateur.

6. Logs structures

6.1 Recommandation

Pour Docker, un format JSON sur stdout devient utile des que les logs doivent etre filters ou centralises. Pour la toute premiere V1, un format texte key/value coherent peut suffire. L'essentiel est de conserver des champs stables.

Schema cible minimal :

```
timestamp
level
logger
service
module
```

operation
event_type
interaction_id
guild_id
actor_id
target_id
duration_ms
error_type

Champs conditionnels uniquement. Ne pas produire des valeurs null partout si le formater peut les omettre.

6.2 Exemple sémantique

level=ERROR
service=claviger
operation=noctis.apply_selection
interaction_id=...
guild_id=...
actor_id=...
error_type=DiscordHTTPException
duration_ms=184

La stack trace reste attachée au log technique mais ne doit pas être recopiée dans chaque champ.

7. Correlation IDs et Request IDs

Discord fournit déjà un identifiant d'interaction. Il peut jouer le rôle de request_id pour une commande ou une soumission de modal.

Pour un workflow ouvert par une commande puis soumis plus tard par une modal, deux interactions Discord différentes existent. Un correlation_id applicatif peut être créé lors de l'ouverture puis transporté dans l'objet modal si la corrélation devient utile.

Recommandation V1 :

- commencer avec interaction_id sur chaque operation;
- n'ajouter correlation_id que lorsqu'un incident nécessite réellement de relier ouverture et submit;
- ne pas introduire de tracing distribué.

8. Rotation et retention

8.1 Strategie recommandee pour Docker sur mini-PC

Application : logs vers stdout/stderr, pas vers un fichier gere par Python.

Docker : limiter la taille du driver de logs. Valeur de depart raisonnable :

- environ 10 Mo par fichier;
- 5 fichiers conserves;
- ajustement apres observation du volume reel.

Cela evite qu'un disque du mini-PC soit rempli par un incident repetitif.

8.2 Retention des reports administratifs

Les IDs Discord constituent des identifiants persistants et le domaine Noctis est sensible. Ne pas conserver indefiniment les historiques de details.

Politique de depart :

- logs techniques locaux : 14 a 30 jours selon volume;
- reports d'incident administratif : conserver le temps utile au diagnostic, puis archiver/supprimer selon les pratiques du serveur;
- ne jamais journaliser la liste des categories adultes choisies par un membre si elle n'est pas necessaire au diagnostic.

9. Monitoring

9.1 Minimum adapte a la V1

Ne pas deployer Prometheus/Grafana uniquement pour paraitre professionnel.

Sur une seule instance :

- log de demarrage avec version/build;
- log on_ready et reconnexion;
- alerte/ERROR si DB non READY lors d'un workflow;
- report des erreurs Discord/persistence;
- compteur ou resume des catalog sync;
- restart policy Docker;
- surveillance de l'espace disque/volume DB au niveau hote.

9.2 Health check

Un health check qui verifie uniquement l'existence du process ne prouve pas que le bot est connecte a Discord. Un check DB seul ne prouve pas non plus la disponibilite Discord.

Pour la V1, preferer :

- processus qui quitte sur erreur fatale;
- restart Docker;
- evenements de readiness/reconnexion dans les logs;
- eventuelle commande admin /claviger status agregant DB + client.is_ready().

Un endpoint HTTP health dedie ne devient utile que si une plateforme de monitoring doit l'interroger.

9.3 Sentry

Sentry peut devenir utile si des erreurs rares restent difficiles a reproduire apres le deploiement. Ce n'est pas un prerequis de la V1. Le projet doit d'abord avoir un logging local correctement configure.

10. Securite des secrets

10.1 Etat actuel

Points positifs observes :

- .env ignore par Git;
- .env.example ne contient aucune valeur reelle;
- bases SQLite ignorees;
- aucun token Discord en clair confirme dans les sources auditees;
- le token n'est pas affiche par le code.

10.2 Recommandations de deploiement

- injecter DISCORD_TOKEN au runtime;
- ne jamais passer le token en argument de ligne de commande;
- ne pas l'inclure dans les images Docker ou logs;
- permissions minimales sur le fichier contenant les variables si un fichier est utilise;
- token de dev distinct de production;
- rotation immediate si un token apparait dans Git/historique/log.

10.3 Redaction

Avant d'envoyer details=str(error) vers un reporter, prevoir une fonction de sanitization pouvant masquer :

- tokens;
- secrets/API keys;
- chemins locaux excessivement precis si non utiles;
- contenu utilisateur;

- cookies/headers si de futures integrations HTTP apparaissent.

11. Permissions et autorisations

11.1 Commandes administratives

Les groupes database, guild, catalog, roles et report verifient le propriétaire du serveur dans le code. C'est simple et restrictif.

A moyen terme, centraliser ce guard evite la duplication, mais sa presence explicite actuelle est preferable a une abstraction mal testee.

11.2 /say

Le systeme de Capability est une bonne fondation.

Risque : la notion de "trusted role" est derivee de la position au-dessus de Claviger. Pour une fonction de messagerie, cela peut etre un compromis acceptable. Pour ban/kick/modification de config/outils IA, ce ne doit pas etre le modele final.

Cible :

role_id + capability + guild_id -> grant explicite

Un futur agent IA doit egalement agir avec l'identite/capabilities d'un actor defini, jamais avec des droits globaux implicites.

11.3 Roles geres

L'executor refuse :

- role absent;
- role Discord managed;
- role au meme niveau ou au-dessus du top role Claviger.

Le discovery catalogue doit appliquer exactement la meme regle.

12. Validation des entrees

12.1 Selections de questionnaire

Les planners refusent une cle inconnue. La soumission reconstruit le questionnaire courant, ce qui protege contre une UI stale.

Bon niveau de validation.

12.2 Metadata catalogue

Discord limite déjà les tailles via TextInput. Les labels/descriptions sont tronqués pour les checkbox options.

A vérifier :

- normalisation raisonnable des emojis;
- caractères de contrôle si les metadata sont réutilisées ailleurs;
- longueur cote DB si une future API externe les consomme.

SQLite n'impose pas ici de taille stricte, mais Discord UI sert actuellement de frontière.

12.3 SQL

Valeurs paramétrées. Les seuls identifiants interpolables (`table_name`, `key_column`) proviennent du code et sont valides par regex. Risque injection SQL faible.

12.4 Noms de role/channel

Ils sont contrôlés par Discord, mais l'identité par nom est ambiguë. Valider l'unicité ou utiliser des IDs.

13. Dependances et supply chain

Etat actuel

pyproject.toml declare :

- `discord.py` >=2.6,<3.0;
- `python-dotenv` >=1.0,<2.0;
- `aiosqlite` >=0.20,<1.0;
- Pytest/Ruff en dev.

Les bornes majeures réduisent certaines ruptures, mais aucune lockfile n'est fournie dans la branche auditee.

Recommandation

Avant Docker production :

1. choisir un mécanisme de lock (`uv.lock`, `requirements compile`, `Poetry`, etc.);
2. versionner ce lock;
3. construire l'image à partir de celui-ci;
4. CI sur Python 3.12;
5. scan périodique des dépendances (`pip-audit` ou `Dependabot`) après mise en place de la CI;

- 6. ne pas mettre a jour automatiquement en production sans passage des tests.

14. Checklist pre-production

Code et erreurs

- ☐ Tous les boundaries Discord logguent les erreurs inattendues.
- ☐ Les ERROR inattendus conservent une stack trace locale.
- ☐ Les messages utilisateurs ne revelent pas de details internes.
- ☐ Les executions partielles de roles sont detectees et reportees.
- ☐ role_manageable est coherent dans discovery et executor.
- ☐ Les doublons de roles fixes sont refuses ou resolus par ID.
- ☐ Les doublons de catalog_key sont detectes.

Logs et reporting

- ☐ Logging configure au demarrage.
- ☐ print remplace par logger.
- ☐ INFO/WARNING/ERROR visibles dans Docker.
- ☐ Reporter Discord considere comme secondaire.
- ☐ Timeout/strategie en cas de reporter lent.
- ☐ Aucune selection adulte detaillee inutilement dans les logs.
- ☐ Redaction des secrets activee.
- ☐ Rotation/limite Docker configuree.

Secrets et configuration

- ☐ Aucun secret versionne.
- ☐ Token production distinct du dev.
- ☐ IDs/config validates au demarrage.
- ☐ Policy de panne fail-closed.
- ☐ allowed_mentions explicitement decide pour /say.

Base de donnees

- ☐ DB dans un volume persistant.
- ☐ Backup manuel teste.
- ☐ Restauration testee.
- ☐ Migration testee sur une copie de DB V1.
- ☐ Le conteneur ne cree pas une DB au mauvais chemin par erreur.

Tests et supply chain

- ☐ Suite complete verte dans l'environnement de build.
- ☐ Ruff vert.
- ☐ Test de failure partielle executor ajoute.

- ☐ Test role managed discovery ajoute.
- ☐ Lockfile versionnee.
- ☐ CI execute tests + lint.

15. Roadmap d'amelioration

Avant production

1. Logging runtime.
2. Error boundaries/reporting uniforme.
3. Tracebacks.
4. Detection des executions partielles Discord.
5. Correction role_manageable.
6. Lock de dependances.
7. Volume DB + backup/restauration Docker.

Apres deploiement immediat

1. IDs/unique resolution pour roles et salons fixes.
2. Doublons de catalog keys.
3. Redaction formelle des details.
4. CI et scan de dependances.
5. Policy de mentions /say.
6. Timeout des reporters distants.

Court terme

1. Unifier executor/plan/resultat.
2. Reorganiser tests.
3. Normaliser nomenclature.
4. Centraliser settings.
5. Centraliser guards/error mapping Discord.

Moyen terme

1. Explicit capability grants.
2. Isolation integrations Discord.
3. Petit tableau de sante/diagnostic.
4. Sentry uniquement si les logs montrent un besoin.

Optionnel / evolution professionnelle

1. CI/CD deploiement mini-PC.
2. ADRs courts sur choix architecturaux.
3. Interface Web.
4. Multi-guild.
5. Tools IA audites et permissionnes.

6. Agent conversationnel avec confirmation des actions sensibles.

Conclusion

La strategie de securisation de Claviger doit rester proportionnee. Le projet n'a besoin ni de Kubernetes ni d'un SIEM pour etre professionnel. Il a besoin de trois garanties simples et fortes :

1. **une action ne doit pas depasser les permissions prevues;**
2. **un echec doit laisser un etat compréhensible et observable;**
3. **les logs doivent permettre de reconstruire un incident sans exposer de secrets ou de contenu sensible.**

La base architecturale existe deja. Le prochain travail consiste surtout a la rendre operationnellement fiable.